Pune Vidyarthi Griha's
COLLEGE OF ENGINEERING, TECHNOLOGY AND MANAGEMENT, PUNE-09

Name : **Class : B.E.** **Roll No :**

PRACTICAL NO. 01

Linear Discriminant Analysis (LDA) Classification on the Iris Dataset

Aim

To implement Linear Discriminant Analysis on the Iris dataset, which finds projections that best separate the classes, and to use it to classify flower species and evaluate the result with accuracy, a confusion matrix and a classification report.

Code and Output

In [1]:
```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             classification_report, ConfusionMatrixDisplay)
import matplotlib.pyplot as plt
```

In [2]:
```
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.25, random_state=42,
    stratify=iris.target
)
model = LinearDiscriminantAnalysis()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Test accuracy: {:.4f}".format(accuracy_score(y_test, y_pred)))
print("\nConfusion matrix (rows=actual, columns=predicted):")
print(confusion_matrix(y_test, y_pred))
print("\nClassification report:")
print(classification_report(y_test, y_pred, target_names=iris.target_names,
                            zero_division=0))
```

```
Test accuracy: 1.0000

Confusion matrix (rows=actual, columns=predicted):
[[12  0  0]
 [ 0 13  0]
 [ 0  0 13]]

Classification report:
              precision    recall  f1-score   support

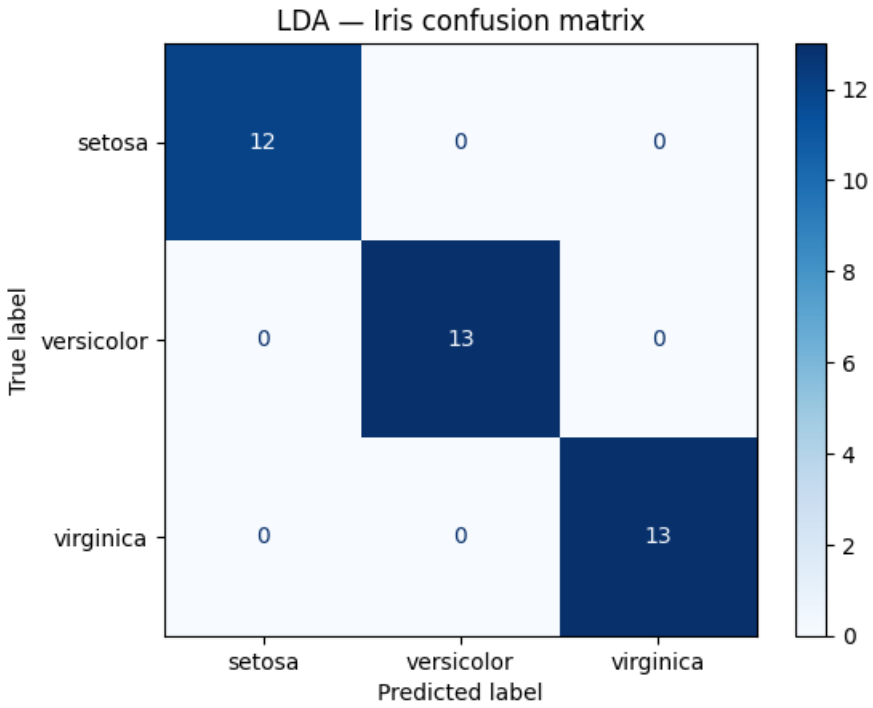
      setosa       1.00      1.00      1.00        12
  versicolor       1.00      1.00      1.00        13
   virginica       1.00      1.00      1.00        13
    accuracy                           1.00        38
```

```
   macro avg       1.00      1.00      1.00        38
weighted avg       1.00      1.00      1.00        38
```

In [3]:

```
ConfusionMatrixDisplay.from_predictions(y_test, y_pred,
                                        display_labels=iris.target_names)
plt.title("LDA - Iris confusion matrix"); plt.show()

sample = [[5.1, 3.5, 1.4, 0.2]]
print("Sample measurements:", sample[0])
print("Predicted species:", iris.target_names[model.predict(sample)[0]])
```




```
Sample measurements: [5.1, 3.5, 1.4, 0.2]
Predicted species: setosa
```

Pune Vidyarthi Griha's
COLLEGE OF ENGINEERING, TECHNOLOGY AND MANAGEMENT, PUNE-09

Name : **Class : B.E.** **Roll No :**

PRACTICAL NO. 02

Uber Fare Prediction using Linear, Ridge and Lasso Regression

Aim

To pre-process the Uber Fares dataset (clean the data and identify fare outliers), inspect the correlations between features, and fit Linear, Ridge and Lasso regression models to predict the fare, comparing them using R^2, RMSE and MAE.

Code and Output

In [1]:
```
import os, numpy as np, pandas as pd, matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
```

In [2]:
```
DATA_PATH = "uber.csv"  # rename your downloaded CSV to this
if not os.path.exists(DATA_PATH):
    raise FileNotFoundError("Place the Uber Fares CSV beside this notebook "
                            "as uber.csv")
df = pd.read_csv(DATA_PATH)
df.columns = [c.strip().lower() for c in df.columns]
display(df.head())
print("Original shape:", df.shape)
```

```
   unnamed: 0                            key  fare_amount  \
0    24238194    2015-05-07 19:52:06.0000003          7.5
1    27835199    2009-07-17 20:04:56.0000002          7.7
2    44984355   2009-08-24 21:45:00.00000061         12.9
3    25894730    2009-06-26 08:22:21.0000001          5.3
4    17610152  2014-08-28 17:47:00.000000188         16.0

           pickup_datetime  pickup_longitude  ...
0  2015-05-07 19:52:06 UTC        -73.999817  ...
1  2009-07-17 20:04:56 UTC        -73.994355  ...
2  2009-08-24 21:45:00 UTC        -74.005043  ...
3  2009-06-26 08:22:21 UTC        -73.976124  ...
4  2014-08-28 17:47:00 UTC        -73.925023  ...

Original shape: (200000, 9)
```

In [3]:
```
target = "fare_amount"
if target not in df.columns:
    raise ValueError(f"Expected target column '{target}'. "
```

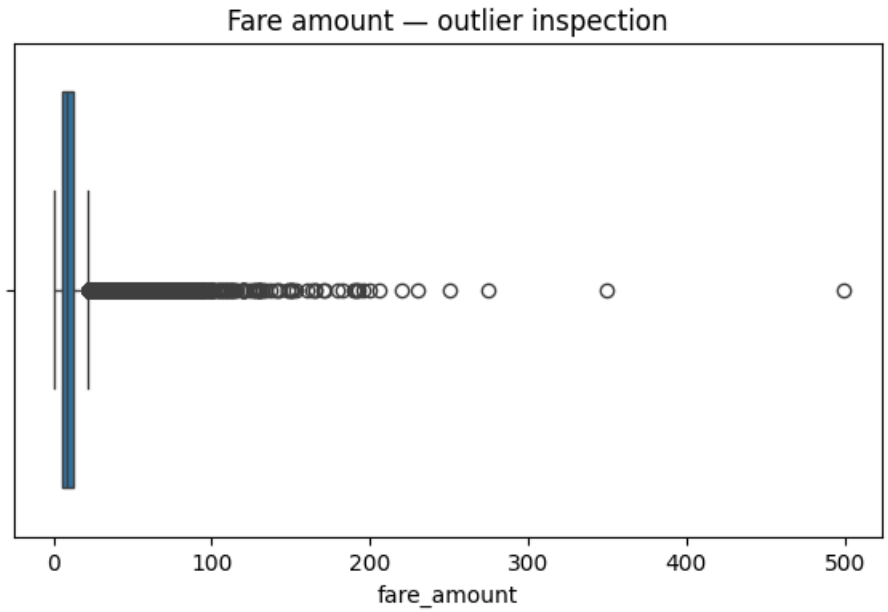
```
                     f"Found: {list(df.columns)}")
df = df.replace([np.inf, -np.inf], np.nan).dropna(subset=[target])
df = df[df[target] >= 0]
for col, lo, hi in [("pickup_longitude", -180, 180),
                    ("dropoff_longitude", -180, 180),
                    ("pickup_latitude", -90, 90),
                    ("dropoff_latitude", -90, 90)]:
    if col in df.columns: df = df[df[col].between(lo, hi)]
if "passenger_count" in df.columns:
    df = df[df["passenger_count"].between(1, 6)]
if "pickup_datetime" in df.columns:
    dt = pd.to_datetime(df["pickup_datetime"], errors="coerce", utc=True)
    df["pickup_hour"] = dt.dt.hour
    df["pickup_day"] = dt.dt.day
    df["pickup_month"] = dt.dt.month
    df["pickup_weekday"] = dt.dt.dayofweek
    df = df.drop(columns=["pickup_datetime"])
df = df.select_dtypes(include="number").dropna(subset=[target])
q1, q3 = df[target].quantile([.25, .75]); iqr = q3 - q1
outlier_mask = (df[target] < q1 - 1.5 * iqr) | (df[target] > q3 + 1.5 * iqr)
print("Rows after basic cleaning:", len(df))
print("Fare outliers by IQR rule:", int(outlier_mask.sum()))
display(df.describe().T)
```

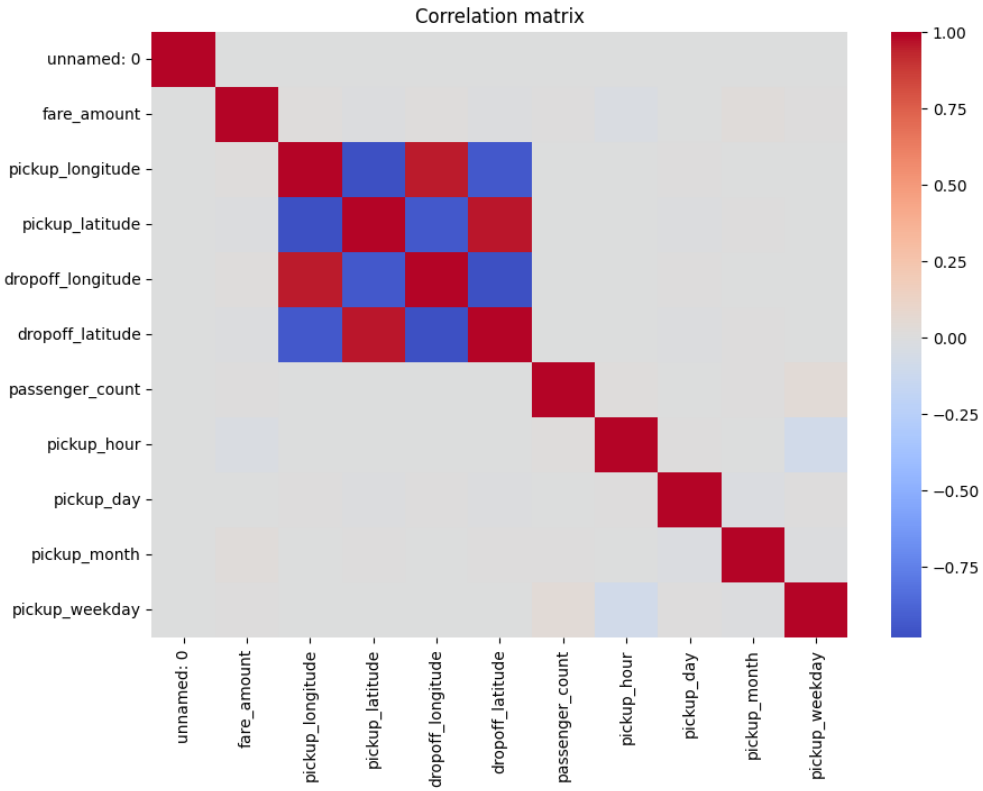
```
Rows after basic cleaning: 199261
Fare outliers by IQR rule: 17120
```

	count	mean	std	min	...
unnamed: 0	199261.0	2.771598e+07	1.601407e+07	1.000000	...
fare_amount	199261.0	1.136909e+01	9.906025e+00	0.000000	...
pickup_longitude	199261.0	-7.250384e+01	1.044338e+01	-93.824668	...
pickup_latitude	199261.0	3.991899e+01	6.128334e+00	-74.015515	...
dropoff_longitude	199261.0	-7.251370e+01	1.040545e+01	-75.458979	...
dropoff_latitude	199261.0	3.992310e+01	6.115539e+00	-74.015750	...
passenger_count	199261.0	1.689453e+00	1.305393e+00	1.000000	...
pickup_hour	199261.0	1.349267e+01	6.516173e+00	0.000000	...
pickup_day	199261.0	1.570564e+01	8.686897e+00	1.000000	...
pickup_month	199261.0	6.283367e+00	3.438473e+00	1.000000	...
pickup_weekday	199261.0	3.049382e+00	1.946785e+00	0.000000	...

In [4]:

```
plt.figure(figsize=(7,4))
sns.boxplot(x=df[target])
plt.title("Fare amount - outlier inspection"); plt.show()
corr = df.corr(numeric_only=True)
plt.figure(figsize=(10,7))
sns.heatmap(corr, cmap="coolwarm", center=0)
plt.title("Correlation matrix"); plt.show()
print("Correlation with fare:")
display(corr[target].sort_values(ascending=False).to_frame("correlation"))
```





Correlation with fare:

	correlation
fare_amount	1.000000
pickup_month	0.023509
dropoff_longitude	0.011899
pickup_longitude	0.011558
passenger_count	0.010173
pickup_weekday	0.007213
pickup_day	0.001439
unnamed: 0	0.000613
pickup_latitude	-0.011485
dropoff_latitude	-0.012282
pickup_hour	-0.021460

In [5]:

```
# Use a sample for faster practical execution on very large CSVs.
if len(df) > 100_000:
    df = df.sample(100_000, random_state=42)
X, y = df.drop(columns=[target]), df[target]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=.2, random_state=42)
models = {"Linear Regression": LinearRegression(), "Ridge": Ridge(alpha=1.0),
          "Lasso": Lasso(alpha=.01, max_iter=10000)}
rows = []
for name, estimator in models.items():
    pipe = Pipeline([("imputer", SimpleImputer(strategy="median")),
                     ("scaler", StandardScaler()), ("model", estimator)])
    pipe.fit(X_train, y_train)
    pred = pipe.predict(X_test)
    rows.append({"Model":name, "R2":r2_score(y_test,pred),
                 "RMSE":np.sqrt(mean_squared_error(y_test,pred)),
                 "MAE":mean_absolute_error(y_test,pred)})
results = pd.DataFrame(rows).sort_values("R2", ascending=False)
display(results.round(4))
```

	Model	R2	RMSE	MAE
2	Lasso	0.0013	10.0735	6.0905
1	Ridge	0.0013	10.0738	6.0910
0	Linear Regression	0.0013	10.0738	6.0910

Pune Vidyarthi Griha's
COLLEGE OF ENGINEERING, TECHNOLOGY AND MANAGEMENT, PUNE-09

Name : **Class : B.E.** **Roll No :**

PRACTICAL NO. 03

Support Vector Machine (SVM) for Handwritten Digit Classification

Aim

To implement a Support Vector Machine with an RBF kernel that classifies handwritten digits (0–9) from the scikit-learn digits dataset of 8×8 images, and to evaluate it using accuracy, a confusion matrix and a classification report.

Code and Output

In [1]:
```
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             classification_report, ConfusionMatrixDisplay)
```

In [2]:
```
digits=load_digits()
X_train,X_test,y_train,y_test=train_test_split(
    digits.data,digits.target,test_size=.25,random_state=42,
    stratify=digits.target)
model=make_pipeline(StandardScaler(),SVC(kernel="rbf",C=5,gamma="scale"))
model.fit(X_train,y_train)
pred=model.predict(X_test)
print("Accuracy:",round(accuracy_score(y_test,pred),4))
print("Confusion matrix:\n",confusion_matrix(y_test,pred))
print(classification_report(y_test,pred,zero_division=0))
```

```
Accuracy: 0.98
Confusion matrix:
 [[45  0  0  0  0  0  0  0  0  0]
 [ 0 45  0  0  1  0  0  0  0  0]
 [ 0  0 44  0  0  0  0  0  0  0]
 [ 0  0  0 46  0  0  0  0  0  0]
 [ 0  0  0  0 44  0  0  1  0  0]
 [ 0  0  0  0  0 45  0  0  0  1]
 [ 0  0  0  0  0  0 45  0  0  0]
 [ 0  0  0  0  0  1  0 44  0  0]
 [ 0  2  0  0  1  0  0  0 40  0]
 [ 0  0  0  0  0  0  1  1  0 43]]
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        45
           1       0.96      0.98      0.97        46
           2       1.00      1.00      1.00        44
```

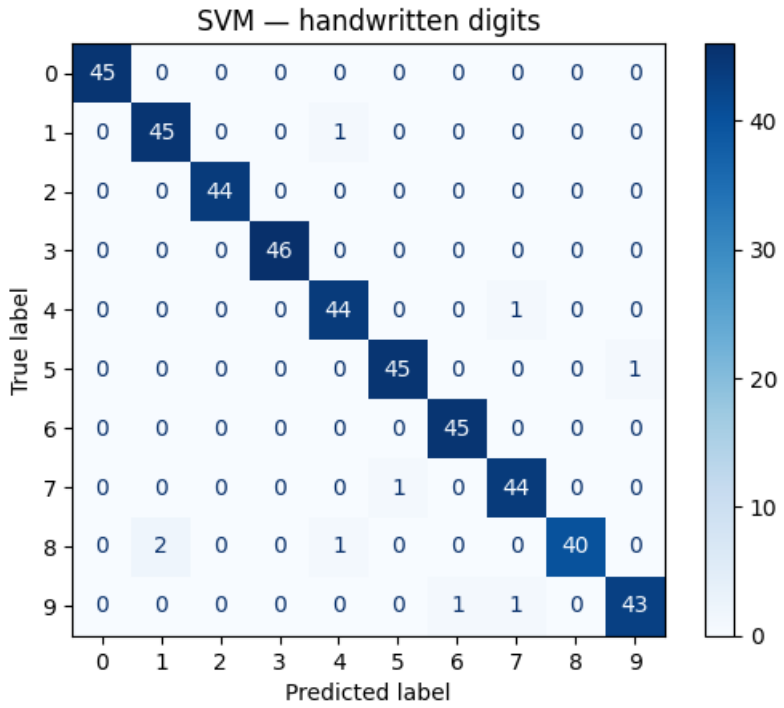
```
              3       1.00      1.00      1.00        46
              4       0.96      0.98      0.97        45
              5       0.98      0.98      0.98        46
              6       0.98      1.00      0.99        45
              7       0.96      0.98      0.97        45
              8       1.00      0.93      0.96        43
              9       0.98      0.96      0.97        45

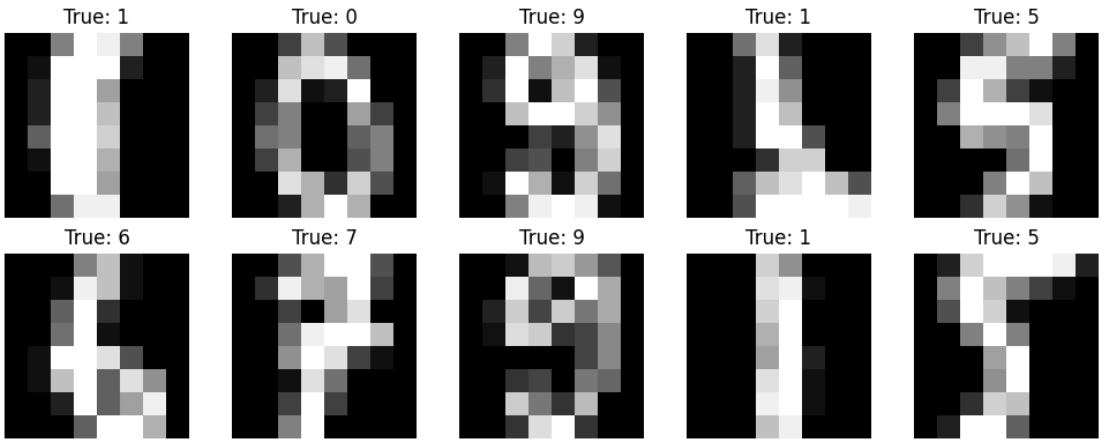
    accuracy                           0.98       450
   macro avg       0.98      0.98      0.98       450
weighted avg       0.98      0.98      0.98       450
```

In [3]:

```
ConfusionMatrixDisplay.from_predictions(y_test,pred,cmap="Blues")
plt.title("SVM - handwritten digits"); plt.show()

# Display first 10 held-out images by original row index
X_train_idx, X_test_idx, y_train2, y_test2 = train_test_split(
    range(len(digits.data)), digits.target, test_size=.25, random_state=42,
    stratify=digits.target)
fig,axes=plt.subplots(2,5,figsize=(10,4))
for ax,idx in zip(axes.ravel(),X_test_idx[:10]):
    ax.imshow(digits.images[idx],cmap="gray")
    ax.set_title(f"True: {digits.target[idx]}"); ax.axis("off")
plt.tight_layout(); plt.show()
```




True: 1
True: 0
True: 9
True: 1
True: 5
True: 6
True: 7
True: 9
True: 1
True: 5

Name : **Class : B.E.** **Roll No :**

PRACTICAL NO. 04

K-Means Clustering on the Iris Dataset using the Elbow Method

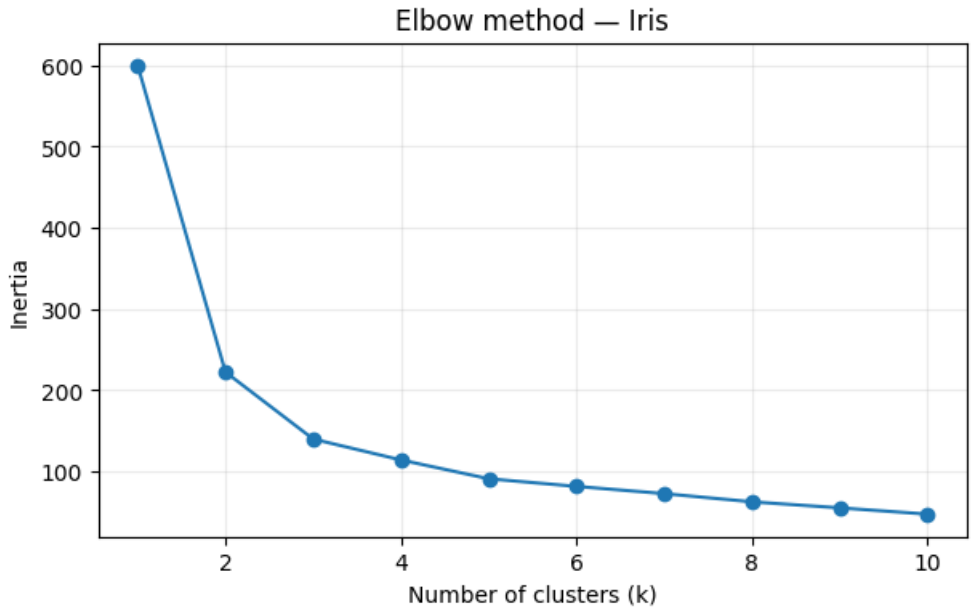
Aim

To implement K-Means clustering on the Iris dataset, use the elbow method on within-cluster inertia to choose the number of clusters k, and evaluate the final clustering with cluster counts and the silhouette score.

Code and Output

In [1]:
```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

In [2]:
```
iris=load_iris()
X=StandardScaler().fit_transform(iris.data)
k_values=range(1,11); inertias=[]
for k in k_values:
    model=KMeans(n_clusters=k,random_state=42,n_init=10).fit(X)
    inertias.append(model.inertia_)
plt.figure(figsize=(7,4)); plt.plot(list(k_values),inertias,marker="o")
plt.xlabel("Number of clusters (k)"); plt.ylabel("Inertia")
plt.title("Elbow method - Iris")
plt.grid(alpha=.25); plt.show()
print("Inertia values:",[round(v,2) for v in inertias])
```

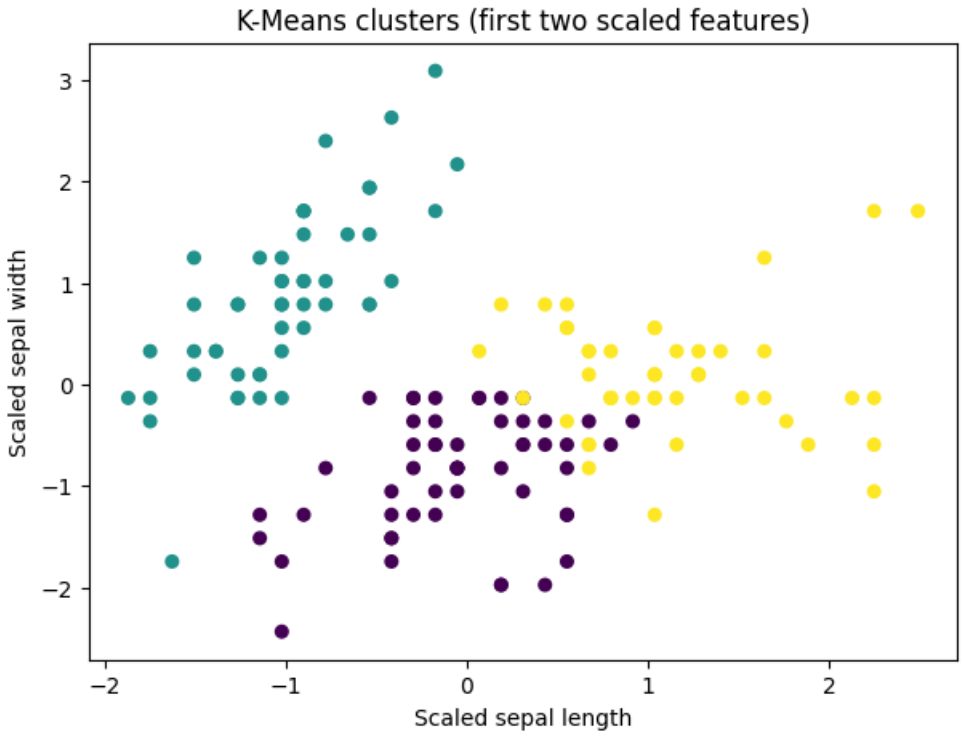



```
Inertia values: [600.0, 222.36, 139.82, 114.09, 90.93, 81.54, 72.63, 62.54, 55.12, 47.39]
```

In [3]:

```python
# Iris has three known species; choose k=3 for the final clustering.
k=3
model=KMeans(n_clusters=k,random_state=42,n_init=10)
labels=model.fit_predict(X)
print("Chosen k:",k)
print("Cluster counts:",{int(i):int((labels==i).sum()) for i in set(labels)})
print("Silhouette score:",round(silhouette_score(X,labels),4))
plt.figure(figsize=(7,5))
plt.scatter(X[:,0],X[:,1],c=labels,cmap="viridis",s=30)
plt.xlabel("Scaled sepal length"); plt.ylabel("Scaled sepal width")
plt.title("K-Means clusters (first two scaled features)"); plt.show()
```

```
Chosen k: 3
Cluster counts: {0: 53, 1: 50, 2: 47}
Silhouette score: 0.4599
```


K-Means clusters (first two scaled features)
Scaled sepal width
Scaled sepal length
3
2
1
0
−1
−2
−2
−1
0
1
2

Name : **Class : B.E.** **Roll No :**

PRACTICAL NO. 05

Ensemble Learning: Voting, AdaBoost, Gradient Boosting and XGBoost on Iris

Aim

To train ensemble classifiers (hard Voting, AdaBoost, Gradient Boosting and XGBoost) on the Iris dataset and compare them using accuracy, macro precision, macro recall and macro F1-score.

Code and Output

In [1]:
```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (VotingClassifier, AdaBoostClassifier,
                              GradientBoostingClassifier,
                              RandomForestClassifier)
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, classification_report)
```

In [2]:
```
iris=load_iris()
X_train,X_test,y_train,y_test=train_test_split(
    iris.data,iris.target,test_size=.25,random_state=42,stratify=iris.target)
models={
 "Voting (hard)":VotingClassifier(estimators=[
     ("lr",make_pipeline(StandardScaler(),LogisticRegression(max_iter=2000))),
     ("dt",DecisionTreeClassifier(max_depth=3,random_state=42)),
     ("rf",RandomForestClassifier(n_estimators=150,random_state=42))],
     voting="hard"),
 "AdaBoost":AdaBoostClassifier(n_estimators=100,random_state=42),
 "Gradient Boosting":GradientBoostingClassifier(random_state=42)
}
try:
    from xgboost import XGBClassifier
    models["XGBoost"]=XGBClassifier(n_estimators=100,max_depth=3,
        learning_rate=.1,objective="multi:softprob",num_class=3,
        eval_metric="mlogloss",random_state=42)
except Exception as e:
    print("XGBoost unavailable; skipped:",e)
```

In [3]:
```
rows=[]
for name,model in models.items():
    model.fit(X_train,y_train); pred=model.predict(X_test)
    rows.append({"Model":name,"Accuracy":accuracy_score(y_test,pred),
```

```
        "Macro precision":precision_score(y_test,pred,average="macro",
                                          zero_division=0),
        "Macro recall":recall_score(y_test,pred,average="macro",
                                    zero_division=0),
        "Macro F1":f1_score(y_test,pred,average="macro",zero_division=0)})
    print("\n",name,"\n",classification_report(y_test,pred,
          target_names=iris.target_names,zero_division=0))
results=pd.DataFrame(rows).sort_values("Accuracy",ascending=False)
display(results.round(4))
```

```
 Voting (hard)
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        12
  versicolor       0.80      0.92      0.86        13
   virginica       0.91      0.77      0.83        13

    accuracy                           0.89        38
   macro avg       0.90      0.90      0.90        38
weighted avg       0.90      0.89      0.89        38


 AdaBoost
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        12
  versicolor       0.86      0.92      0.89        13
   virginica       0.92      0.85      0.88        13

    accuracy                           0.92        38
   macro avg       0.92      0.92      0.92        38
weighted avg       0.92      0.92      0.92        38


 Gradient Boosting
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        12
  versicolor       1.00      0.92      0.96        13
   virginica       0.93      1.00      0.96        13

    accuracy                           0.97        38
   macro avg       0.98      0.97      0.97        38
weighted avg       0.98      0.97      0.97        38


 XGBoost
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        12
  versicolor       0.86      0.92      0.89        13
   virginica       0.92      0.85      0.88        13

    accuracy                           0.92        38
   macro avg       0.92      0.92      0.92        38
weighted avg       0.92      0.92      0.92        38

              Model  Accuracy  Macro precision  Macro recall  Macro F1
2  Gradient Boosting    0.9737           0.9762        0.9744    0.9743
1           AdaBoost    0.9211           0.9246        0.9231    0.9230
3            XGBoost    0.9211           0.9246        0.9231    0.9230
0      Voting (hard)    0.8947           0.9030        0.8974    0.8968
```

Name : **Class : B.E.** **Roll No :**

PRACTICAL NO. 06

Q-Learning for Path Finding in a Maze

Aim

To implement the Q-Learning reinforcement learning algorithm in which an agent learns action values from rewards in a grid maze, and then follows the learned Q-values to find a path from the start S to the goal G.

Code and Output

In [1]:
```
import numpy as np, random
random.seed(42); np.random.seed(42)
maze=["S....",".##..","...#.","##...","....G"]
ROWS,COLS=len(maze),len(maze[0]); start=(0,0); goal=(4,4)
states=[(r,c) for r in range(ROWS) for c in range(COLS) if maze[r][c]!="#"]
state_id={s:i for i,s in enumerate(states)}
actions=[(-1,0),(1,0),(0,-1),(0,1)]
action_names=["Up","Down","Left","Right"]
Q=np.zeros((len(states),len(actions)))
alpha,gamma,epsilon=.2,.95,.25
def step(s,a):
    dr,dc=actions[a]; ns=(s[0]+dr,s[1]+dc)
    if not (0<=ns[0]<ROWS and 0<=ns[1]<COLS) or maze[ns[0]][ns[1]]=="#":
        ns=s
    done=ns==goal
    return ns,(100 if done else -1),done
episodes=5000
for ep in range(episodes):
    s=start
    for _ in range(100):
        if random.random()<epsilon: a=random.randrange(4)
        else: a=int(np.argmax(Q[state_id[s]]))
        ns,reward,done=step(s,a)
        target=reward+gamma*(0 if done else np.max(Q[state_id[ns]]))
        Q[state_id[s],a]+=alpha*(target-Q[state_id[s],a])
        s=ns
        if done: break
    epsilon=max(.02,epsilon*.999)
print("Training episodes:",episodes)
print("Q-table shape:",Q.shape)
```

```
Training episodes: 5000
Q-table shape: (20, 4)
```

In [2]:
```
s=start; path=[s]
for _ in range(40):
```

```
    if s==goal: break
    # Select a valid move with the highest learned Q-value.
    candidates=[]
    for a in range(4):
        ns,_,_=step(s,a)
        if ns!=s: candidates.append((Q[state_id[s],a],ns))
    if not candidates: break
    s=max(candidates,key=lambda item:item[0])[1]
    path.append(s)
    if s==goal: break
print("Maze:")
for row in maze: print(row)
print("Learned path:", " -> ".join(map(str,path)))
print("Reached goal:",s==goal,"| Steps:",len(path)-1)
```

```
Maze:
S....
.##..
...#.
##...
....G
Learned path: (0, 0) -> (0, 1) -> (0, 2) -> (0, 3) -> (1, 3) -> (1, 4) -> (2, 4)
-> (3, 4) -> (4, 4)
Reached goal: True | Steps: 8
```